SMART RECRUIT

How to Run the Project & End-to-End User Journey

Step-by-Step Setup, Login Credentials, Role Flows and Troubleshooting

Prepared from the inspected project archive

Smart-Team-Project-Test.zip

.NET 8	SQL Server	HTML/CSS/JS	3 Roles
ASP.NET Core	LocalDB	Single App	Admin / Employer / Seeker

Document Date: 11 August 2026

Document Contents

Source-of-truth rule
This guide is based on the current project code and configuration. Where the source does not contain a fixed value (especially demo credentials), the document explicitly says so instead of inventing an existing account.

1. Purpose and Project Run Model

Smart Recruit is a single ASP.NET Core web application. The same .NET process hosts the REST API and the static HTML/CSS/JavaScript frontend. SQL Server LocalDB stores relational data, while CVs and employer-verification files are stored in server-side folders.

Browser -> ASP.NET Core Host -> REST Controllers -> Services / Repositories -> SQL Server LocalDB

No separate frontend server required
There is no React/Vue/Angular development server and no npm start step. ASP.NET Core serves files from wwwroot using UseDefaultFiles() and UseStaticFiles().

Project item	Actual value
Target framework	.NET 8 (net8.0)
Solution / project	Recruitment-Project.slnx / Recruitment-Project.csproj
Database	RecruitmentProjectDb
Database server	(localdb)\MSSQLLocalDB
HTTPS launch URL	https://localhost:7175
HTTP launch URL	http://localhost:5062
IIS Express HTTPS	https://localhost:44366
Swagger launch path	/swagger
Frontend entry point	/ -> wwwroot/index.html

Project item	Actual value
JWT lifetime	60 minutes

2. Prerequisites

Requirement	Why it is needed	Recommended check
Windows 10/11	The configured database uses SQL Server LocalDB, which is Windows-oriented.	Run the project on the team Windows development machine.
.NET 8 SDK	Required by TargetFramework net8.0.	Command Prompt: dotnet --version
Visual Studio 2022	Easiest run/debug experience.	Install ASP.NET and web development workload.
SQL Server Express LocalDB	Required by the configured DefaultConnection.	Check (localdb)\MSSQLLocalDB in SQL Server Object Explorer.
Web browser	Runs the frontend and Swagger UI.	Edge / Chrome / Firefox.
Optional: SSMS	Useful for viewing data and creating the local Administrator role.	Connect to (localdb)\MSSQLLocalDB.

2.1 What You Do NOT Need

- Node.js or npm is not required for this build.
- No separate frontend framework setup is required.
- No external Google Meet API key is required; meeting links are stored as normal links.
- No separate web server such as Apache/Nginx is required for local development.

3. First-Time Project Setup

Extract ZIP -> Open Project -> Restore NuGet -> Create/Update DB -> Run HTTPS Profile

3.1 Extract and Open

1. Extract Smart-Team-Project-Test.zip to a normal writable folder, for example C:\Projects\Smart-Team-Project-Test.
2. Open Recruitment-Project.slnx in Visual Studio. If your Visual Studio does not open the .slnx format, open Recruitment-Project.csproj directly.
3. Allow Visual Studio to finish project loading and NuGet restore before running migrations.

3.2 Confirm the Database Connection

The current appsettings.json contains this LocalDB connection string:

```
Server=(localdb)
\MSSQLLocalDB;Database=RecruitmentProjectDb;Trusted_Connection=True;MultipleActiveResultSets=true;Trust
ServerCertificate=True
```

Do not change it unnecessarily
If all team members use Windows + LocalDB, keep the existing connection string. Change it only when using a different SQL Server instance.

3.3 Apply Entity Framework Core Migrations

The application does not automatically call Database.Migrate() on startup. Therefore a fresh machine must apply the included migrations before the first run.

Visual Studio method:

4. Open Tools -> NuGet Package Manager -> Package Manager Console.
5. Ensure the default project is Recruitment-Project.
6. Run the command below.

```
Update-Database
```

Expected result: SQL Server LocalDB creates or updates the database named RecruitmentProjectDb and applies the repository migrations.

3.4 Verify the Database

7. Open View -> SQL Server Object Explorer in Visual Studio (or SSMS).
8. Expand SQL Server -> (localdb)\MSSQLLocalDB -> Databases.
9. Confirm RecruitmentProjectDb exists.
10. Confirm tables such as Users, JobSeekerProfiles, EmployerProfiles, Vacancies, JobApplications, Notifications and JobReports exist.

4. Run the Application

4.1 Recommended: Visual Studio

11. In the toolbar, choose the https launch profile.
12. Press F5 for debugging or Ctrl+F5 to run without debugging.
13. Accept/trust the ASP.NET development HTTPS certificate if Windows asks.
14. The launch profile opens Swagger automatically because launchUrl is set to swagger.
15. Open the root site separately to use the frontend.

Purpose	URL
Frontend home	https://localhost:7175/
Login	https://localhost:7175/login.html
Register	https://localhost:7175/register.html
Swagger / API testing	https://localhost:7175/swagger

4.2 Alternative: .NET CLI

Use this method if you prefer Command Prompt / PowerShell and have .NET 8 installed:

```
cd C:\Projects\Smart-Team-Project-Test

dotnet restore

# Install once if dotnet-ef is not already available
dotnet tool install --global dotnet-ef --version 8.0.8

dotnet ef database update

dotnet dev-certs https --trust

dotnet run --launch-profile https
```

If dotnet-ef is already installed
Skip the install command. If its major version is incompatible, update it to an EF Core 8 compatible version before running migrations.

5. Successful Startup - What You Should See

16. The console/Visual Studio Output shows the ASP.NET application listening on HTTPS/HTTP URLs.
17. Swagger loads at /swagger in Development environment.
18. The frontend loads from / and displays the Smart Recruit landing page.
19. Login and Register pages load without a separate frontend process.
20. A request to an authenticated page without login redirects/blocks according to the frontend guards.

First run with a fresh database
The database contains schema only. This repository does not seed demo users, jobs, employers or an Administrator account, so empty dashboards are normal until test data is created.

6. Required Login Credentials and Account Setup

Important finding
No fixed or hard-coded Job Seeker, Employer or Administrator credentials were found in the current source. AuthService creates Job Seeker/Employer users only when someone registers. Public Administrator registration is not implemented.

For a repeatable local demo, use the following TEST credentials and create them using the steps in this section. These are local demonstration accounts, not credentials already embedded in the ZIP.

Role	Local demo email	Local demo password	How to create
Administrator	admin.demo@smartrecruit.local	Admin@12345	Register as Employer once, then change Users.Role to 1 in LocalDB.
Employer	employer.demo@smartrecruit.local	Employer@123	Register normally from register.html as Employer.
Job Seeker	jobseeker.demo@smartrecruit.local	JobSeeker@123	Register normally from register.html as Job Seeker.

Security note
Use these only for local demonstration/testing. Do not use shared demo passwords in a deployed production environment.

6.1 Create the Job Seeker Demo Account

21. Open https://localhost:7175/register.html.
22. Choose Job Seeker.
23. Full name: Demo Job Seeker.
24. Email: jobseeker.demo@smartrecruit.local.
25. Password and Confirm Password: JobSeeker@123.
26. Submit registration, then log in from /login.html.

Expected redirect: /module2-jobseeker/dashboard.html

6.2 Create the Employer Demo Account

27. Open /register.html.
28. Choose Employer.
29. Full name: Demo Employer.
30. Email: employer.demo@smartrecruit.local.
31. Password and Confirm Password: Employer@123.
32. Submit registration, then log in.

Expected redirect: /module3-employer/employer-dashboard.html

Employer first-login behavior

Employer registration creates a User account, but it does not automatically create EmployerProfile. Open Company Profile and save the company details before using verification/vacancy flows that depend on EmployerProfile.

6.3 Create the Administrator Demo Account on a Fresh Database

Because the backend intentionally has no public Administrator registration endpoint, use this one-time LOCAL development setup:

33. Register admin.demo@smartrecruit.local from /register.html as an Employer using password Admin@12345.
34. Open SQL Server Object Explorer or SSMS and connect to (localdb)\MSSQLLocalDB.
35. Open a query against RecruitmentProjectDb.
36. Run the SQL below.
37. Log out / clear the old browser session, then log in again with the Administrator credentials.

```
USE [RecruitmentProjectDb];

UPDATE [Users]
SET [Role] = 1,
    [IsActive] = 1,
    [UpdatedAt] = GETUTCDATE()
WHERE [Email] = 'admin.demo@smartrecruit.local';

SELECT [Id], [FullName], [Email], [Role], [IsActive]
FROM [Users]
WHERE [Email] = 'admin.demo@smartrecruit.local';
```

Why Role = 1?
The current UserRole enum is Administrator = 1, Employer = 2, JobSeeker = 3. The login service reads the role from the database and places it into the new JWT token.

If Admin still opens the Employer dashboard
Log out and sign in again after changing the database role. The JWT contains the role claim, so a token issued before the SQL update still carries the previous Employer role.

7. Easy Role-by-Role User Journey

7.1 Job Seeker Journey

Register -> Login -> Complete Profile -> Upload CV -> Find Jobs -> View Match -> Apply -> Track Application

Step	Page / Action	Expected result
1	/register.html -> Job Seeker	Creates User + JobSeekerProfile.
2	/login.html	JWT session created; redirects to Job Seeker dashboard.
3	Profile	Add professional title, location, experience, education, about and skills.
4	Profile -> CV	Upload PDF/DOC/DOCX up to 5 MB.
5	Find Jobs	Browse only vacancies that are Open.
6	Job Details	See backend-calculated match score, skill breakdown, matched/missing skills and trust label.
7	Apply Now	Optional cover letter; duplicate application is

Step	Page / Action	Expected result
		blocked.
8	My Applications	Track Applied / UnderReview / Shortlisted / Rejected.
9	Contact Requests	Accept or reject employer contact request.
10	Interviews / Notifications	View scheduled interviews and generated notifications.
11	Report Job (optional)	Report an open vacancy and track report status.

Matching behavior
The score is calculated by the backend. Current MatchingService weights skills up to 60 points, experience 20, education 10 and exact location match 10. The browser should display the result, not calculate it.

7.2 Employer Journey

Register -> Login -> Company Profile -> Verification -> Create Vacancy -> Submit -> Review Applicants -> Contact / Interview

Step	Page / Action	Expected result
1	Register as Employer	Creates the User account.
2	Company Profile	Create/save EmployerProfile before dependent features.
3	Verification	Upload PDF/JPG/JPEG/PNG up to 5 MB and submit for Admin review. Current backend requires at least one document.
4	Vacancy Form	Create a Draft vacancy with title, location, description, required skills, expiry date and other fields.
5	Submit Vacancy	If verified -> Open. If unverified -> PendingApproval.
6	Applicants	Select a vacancy and review application status/match score.
7	Status	Move application through valid statuses such as UnderReview or Shortlisted, or reject.
8	Contact Request	Send a message to the Job Seeker.
9	Interview	Schedule a future interview and optional meeting link/instructions.
10	Notifications	Receive application/contact-response events.

7.3 Administrator Journey

Login -> Dashboard -> Users -> Employer Verification -> Pending Vacancies -> Job Reports -> Moderation / Audit

Step	Page / Action	Expected result
1	Admin login	Redirects to /module1-core/admin-dashboard.html.

Step	Page / Action	Expected result
2	Dashboard	View total users, Job Seekers, Employers, active and disabled users.
3	Users	View users and enable/disable accounts; current admin cannot disable self.
4	Employer Verifications	Verify, reject or request more information for submitted requests.
5	Pending Vacancies	Approve or reject vacancies submitted by unverified employers.
6	Reported Jobs	Review Job Seeker reports, start review, resolve or dismiss.
7	Moderation	Warn/suspend/disable employers or close vacancies with decision notes.
8	Moderation Audit	Review read-only audit history.

8. Recommended Full End-to-End Demo / Test Journey

For a presentation or evaluator test, the following order creates the necessary data dependencies and demonstrates the integration between all five modules.

Phase	Login as	Do this	Why / expected state
A	Administrator	Create the local admin account using Section 6.3.	Enables all administrative flows.
B	Employer	Register Employer and complete Company Profile.	EmployerProfile is required for verification/vacancies.
C	Employer	Create a Draft vacancy and submit it before employer verification.	It should become PendingApproval because employer is unverified.
D	Administrator	Open Pending Vacancies and approve the submitted vacancy.	Vacancy becomes Open and can be found by Job Seekers.
E	Employer	Upload at least one verification document and submit verification.	Verification becomes PendingReview.
F	Administrator	Open Employer Verifications and verify the employer.	Future submitted vacancies can become Open directly.
G	Job Seeker	Register, complete profile/skills and upload CV.	Creates matching data and Job Seeker identity.
H	Job Seeker	Find the approved vacancy, open details and apply.	Backend calculates and stores match score; Employer is notified.
I	Employer	Open Applicants, move status to UnderReview/Shortlisted and send Contact Request.	Job Seeker receives status/contact notifications.
J	Job Seeker	Open Contact Requests and accept.	Employer receives the response notification.
K	Employer	Schedule a future interview.	Job Seeker receives interview notification and sees it in Interviews.
L	Job Seeker	Optionally report the vacancy.	Creates a JobReport for Module 5 testing.
M	Administrator	Review the report and use moderation only when appropriate.	Demonstrates reporting, decisions and audit trail.

Conditional vacancy publication
Verified Employer: Submit -> Open directly. Unverified Employer: Submit -> PendingApproval -> Administrator approves/rejects. This is why the recommended demo intentionally submits the first vacancy before employer verification.

9. Important Current-Build Notes

Area	Current code behavior	What the tester should do
Seed data	No users/jobs are seeded.	Create local demo data using this guide.
Admin registration	No public Administrator registration endpoint.	Use the one-time local role update in Section 6.3.
Database startup	Migrations are not auto-applied.	Run Update-Database / dotnet ef database update first.
Employer registration	Does not create EmployerProfile automatically.	Save Company Profile after first Employer login.
Verification documents	Backend currently checks for at least one uploaded document, not three mandatory document types.	Upload at least one supported file for the current build.
Job search minimum-match filter	Request DTO contains minimumMatch but SearchJobsAsync does not currently apply it.	Do not treat minimum-match filtering as a validated current feature.
Interview workflow	Current controller exposes create/list only.	Do not expect accept/decline/reschedule/cancel/complete endpoints in this build.
Interview prerequisite	Current service blocks rejected applications and past dates, but does not enforce accepted contact request.	Follow contact-request acceptance as the intended user journey, while recognizing current backend enforcement is looser.
Password reset	ResetPasswordAsync is intentionally disabled without verified reset token flow.	Do not demo password reset.
JWT expiry	Configured for 60 minutes.	If 401 occurs after time passes, sign in again.

10. Troubleshooting Guide

Problem	Likely cause	Fix
"Cannot open database" / LocalDB connection error	LocalDB missing or not running.	Install SQL Server Express LocalDB; verify (localdb)\MSSQLLocalDB.
"Invalid object name" / table missing	Migrations not applied.	Run Update-Database in Package Manager Console.
Swagger works but frontend not obvious	Launch URL is configured as swagger.	Open https://localhost:7175/ manually.
Browser HTTPS warning	Development certificate not trusted.	Trust Visual Studio prompt or run dotnet dev-certs https --trust.
401 Unauthorized	Not logged in, token expired, or token cleared.	Log in again. JWT lifetime is 60 minutes.
403 / wrong dashboard	Account role does not match page.	Use the correct role; for admin verify Users.Role = 1 and re-login.
Admin still treated as Employer after SQL update	Old JWT still stores Employer claim.	Logout, clear localStorage if necessary, login again.
Employer dashboard/profile features fail	EmployerProfile has not been created.	Open Company Profile and save it first.
No jobs appear for Job Seeker	No Open vacancy exists.	Approve an unverified employer vacancy or submit from a verified employer.
Apply fails	Vacancy closed/expired, employer suspended/disabled, or duplicate application.	Use an Open, non-expired vacancy and a new Job Seeker application.

Problem	Likely cause	Fix
CV upload fails	Unsupported file or >5 MB.	Use PDF, DOC or DOCX up to 5 MB.
Verification upload fails	Unsupported file or >5 MB.	Use PDF, JPG, JPEG or PNG up to 5 MB.
Interview creation fails	Rejected application or date not in the future.	Use non-rejected application and future date/time.

11. Demo-Day / Submission Quick Checklist

- Project extracted to a writable folder.
- .NET 8 SDK and ASP.NET workload installed.
- SQL Server LocalDB available.
- NuGet packages restored.
- RecruitmentProjectDb created with Update-Database.
- HTTPS profile selected and application starts successfully.
- Frontend home loads at https://localhost:7175/.
- Swagger loads at https://localhost:7175/swagger.
- Administrator local demo account created and Role = 1 verified.
- Employer profile created.
- At least one Open vacancy exists.
- Job Seeker profile/skills created for meaningful match score.
- All three demo logins tested before presentation.
- Use a future expiry date for vacancies and future interview date/time.

Fastest presentation sequence
Admin login -> show dashboard. Employer login -> show company/vacancy. Job Seeker login -> show profile, job details + match, apply. Employer -> applicants/contact/interview. Job Seeker -> notifications. Admin -> reports/moderation/audit.

Appendix A - Main Frontend Paths

Role	Page	Path
Public	Home	/
Public	Login	/login.html
Public	Register	/register.html
Job Seeker	Dashboard	/module2-jobseeker/dashboard.html
Job Seeker	Profile	/module2-jobseeker/profile.html
Job Seeker	Jobs	/module2-jobseeker/jobs.html
Job Seeker	Applications	/module4-applications/jobseeker-applications.html
Job Seeker	Contact Requests	/module4-applications/jobseeker-contact-requests.html
Job Seeker	Interviews	/module4-applications/jobseeker-interviews.html
Job Seeker	Notifications	/module5-trust/notifications.html
Job Seeker	Job Reports	/module5-trust/job-reports.html
Employer	Dashboard	/module3-employer/employer-dashboard.html
Employer	Company Profile	/module3-employer/company-profile.html
Employer	Verification	/module3-employer/verification.html
Employer	Vacancies	/module3-employer/vacancies.html

Role	Page	Path
Employer	Applicants	/module4-applications/employer-applicants.html
Employer	Contact Requests	/module4-applications/employer-contact-requests.html
Employer	Interviews	/module4-applications/employer-interviews.html
Administrator	Dashboard	/module1-core/admin-dashboard.html
Administrator	Users	/module1-core/users.html
Administrator	Employer Verifications	/module3-employer/admin-employer-verifications.html
Administrator	Pending Vacancies	/module3-employer/admin-pending-vacancies.html
Administrator	Reported Jobs	/module5-trust/admin-reported-jobs.html
Administrator	Moderation Audit	/module5-trust/moderation-audit.html

Appendix B - Credential Summary

Role	Email	Password	Status
Administrator	admin.demo@smartrecruit.local	Admin@12345	Local demo account must be created + Role changed to 1.
Employer	employer.demo@smartrecruit.local	Employer@123	Create from Register page.
Job Seeker	jobseeker.demo@smartrecruit.local	JobSeeker@123	Create from Register page.

Final reminder
These credentials are a documented local test convention created for a fresh database. They are not hard-coded accounts shipped inside the current source archive.